

Universidad nacional Jorge Basadre Grohmann
Facultad de ingeniería
Escuela profesional de ingeniería en informática y sistemas



Practica N°04

GESTIÓN DE HILOS

Asignatura

Sistemas Operativos

Docente

Ing. Hugo Manuel Barraza Vizcarra

Estudiante

Hugo Antonio Ordoño Quispe

2024-119049

Ciclo: 6to | **Turno:** Mañana

Tacna – Perú

2026

1. Desarrollo

1.1. Creacion y gestion hilos en C.

```
Hago@Hago-VirtualBox: ~/Sistemas_operativos/uso_
Hola desde el hilo 1
Hola desde el hilo 2
Finalizo la ejecucion del programa principal

real    0m0,002s
user    0m0,000s
sys     0m0,002s
```

1.2. Implementación de sincronización

```
Hago@Hago-VirtualBox: ~/Sistemas_operativos/uso_
Hola desde el hilo 1
Hola desde el hilo 2
Finalizo la ejecucion del programa principal

real    0m0,002s
user    0m0,002s
sys     0m0,000s
```

1.3. Análisis de Rendimiento

```
ejecucion con 1 hilo...
Hola desde el hilo 0
tiempo total de ejecucion con 1 hilos: 0.000455 segundo
ejecucion con 5 hilo...
Hola desde el hilo 0
Hola desde el hilo 1
Hola desde el hilo 2
Hola desde el hilo 4
Hola desde el hilo 3
tiempo total de ejecucion con 5 hilos: 0.000399 segundo
ejecucion con 10 hilo...
Hola desde el hilo 0
Hola desde el hilo 3
Hola desde el hilo 4
Hola desde el hilo 5
Hola desde el hilo 1
Hola desde el hilo 7
Hola desde el hilo 6
Hola desde el hilo 8
Hola desde el hilo 2
Hola desde el hilo 9
tiempo total de ejecucion con 10 hilos: 0.000723 segundo
Finalizo la ejecucion de programa principal
```

1.4. Análisis de resultados

Al comparar el rendimiento de un programa multihilo con un monohilo, ¿qué diferencias observaste en el tiempo de ejecución? ¿Cómo influye la gestión adecuada de hilos en el rendimiento general del programa?

Respecto a las diferencias, al comparar 1 hilo con 5 hilos hay una leve mejora, pero en la comparación de 1 hilo y 10 hay un aumento de tiempo, con esto hay una discrepancia ya que teóricamente debería mejorar y no lo contrario.

Respecto a la gestión de adecuada. Una gestión adecuada permite aprovechar la concurrencia hasta cierto punto. No obstante, si se crean demasiados hilos para tareas muy pequeñas (como un solo printf), la sobrecarga que genera el sistema operativo para crearlos y destruirlos anula cualquier beneficio de velocidad.

¿Qué funciones de la biblioteca pthread utilizaste en tu implementación? Explica brevemente la finalidad de al menos tres de estas funciones y su importancia en la gestión de hilos.

pthread_create: Utilizada en el ciclo for para inicializar y lanzar concurrentemente cada uno de los hilos independientes asignándoles la función hilo_funcion2.

pthread_join: Empleada para asegurar que el hilo principal (main) espere la finalización de todos los hilos creados antes de detener el cronómetro y destruir el mutex.

pthread_mutex_lock y pthread_mutex_unlock: Utilizadas dentro de la función del hilo para garantizar la exclusión mutua, permitiendo que solo un hilo a la vez acceda a la sección crítica (la pantalla de impresión).

¿Cómo se comportó el programa al aumentar el número de hilos más allá de un cierto punto? ¿Hubo un límite en el que el rendimiento no mejoró o incluso disminuyó? Explica por qué esto puede ocurrir en un entorno multihilo.

El programa alcanzó su punto óptimo con 5 hilos. Al pasar a 10 hilos, el rendimiento disminuyó drásticamente en lugar de mejorar, esto debido a que en un entorno multihilo, cada hilo compite por el mismo mutex para poder imprimir. Con 10 hilos, la contención del bloqueo es muy alta: los hilos pasan más tiempo bloqueados haciendo fila que ejecutando código útil. Además, el procesador sufre por el cambio de contexto, gastando recursos en pausar y reanudar hilos continuamente.

Durante la ejecución, ¿observaste alguna condición de carrera o comportamiento inesperado? Si es así, ¿cómo impactó esto en los resultados obtenidos y qué medidas se podrían tomar para prevenirlo?

No se observó ninguna condición de carrera. A pesar de que los hilos terminaron en un orden aleatorio por ejemplo, en la prueba de 5 hilos el orden fue 0, 1, 2, 4, 3, los mensajes impresos salieron completos.

El uso de pthread_mutex_t previno con éxito que los caracteres de diferentes hilos se mezclaran o corrompieran en la terminal. El mutex forzó un acceso serializado a la función printf, protegiendo la consistencia de los resultados visuales a costa de un mayor tiempo de espera cuando el volumen de hilos se elevó a 10.